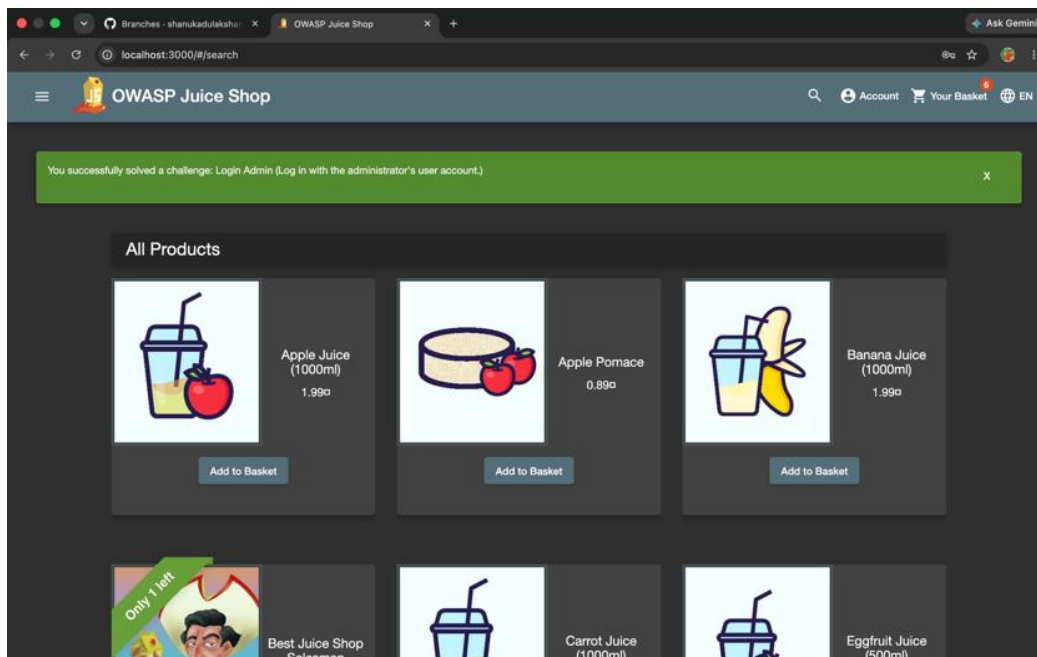
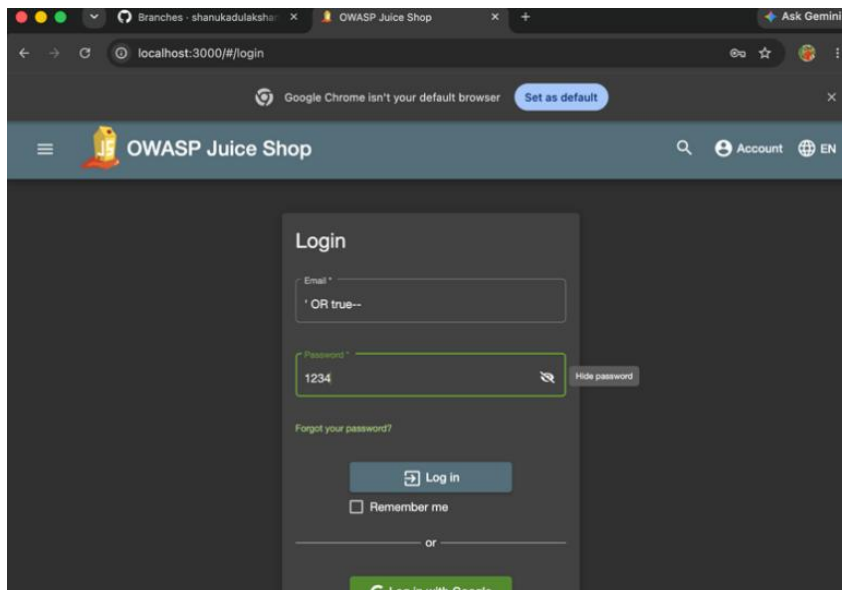
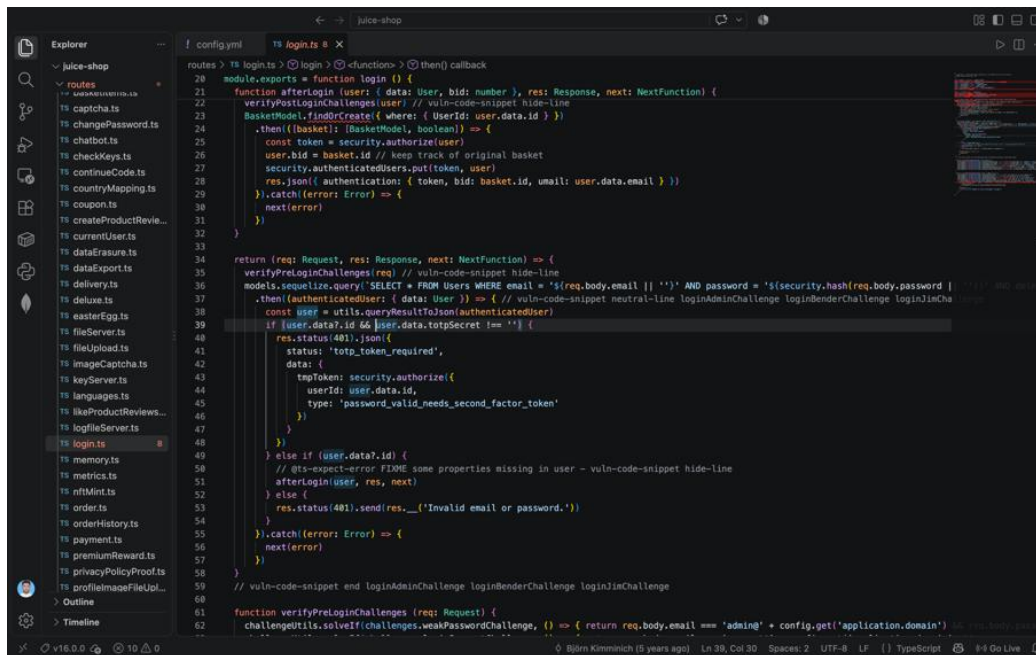


Understanding SQL Injection

Secure Coding Evidence



Scanning Before Fix – SAST SQL Injection Detection



```
routes > TS login.ts > login > <function> > then() callback
20 module.exports = function login () {
21   function afterLogin (user: { data: User, bid: number }, res: Response, next: NextFunction) {
22     verifyPostLoginChallenges(user) // vuln-code-snippet hide-line
23     BasketModel.findOneOrCreate({ where: { UserId: user.data.id } })
24     .then((basket: { BasketModel, boolean }) => {
25       const token = security.authorize(user)
26       user.bid = basket.id // keep track of original basket
27       security.authenticatedUsers.put(token, user)
28       res.json({ authentication: { token, bid: basket.id, email: user.data.email } })
29     }).catch((error: Error) => {
30       next(error)
31     })
32   }
33
34   return (req: Request, res: Response, next: NextFunction) => {
35     verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
36     models.sequelize.query('SELECT * FROM Users WHERE email = \'' + req.body.email + '\'' AND password = \'' + security.hash(req.body.password) + '\'' AND deviceToken = \'' + req.body.deviceToken + '\')
37     .then((authenticatedUser: { data: User }) => { // vuln-code-snippet neutral-line loginAdminChallenge loginBenderChallenge loginJinChallenge
38       const user = utils.queryResultToJson(authenticatedUser)
39       if (user.data.id && user.data.totalSecret !== '') {
40         res.status(401).json({
41           status: 'totp_token_required',
42           data: {
43             tapToken: security.authorize({
44               userId: user.data.id,
45               type: 'password_valid_needs_second_factor_token'
46             })
47           }
48         })
49       } else if (user.data.id) {
50         // @ts-expect-error FIXME some properties missing in user - vuln-code-snippet hide-line
51         afterLogin(user, res, next)
52       } else {
53         res.status(401).send(res._('Invalid email or password.'))
54       }
55     }).catch((error: Error) => {
56       next(error)
57     })
58   }
59   // vuln-code-snippet end loginAdminChallenge loginBenderChallenge loginJinChallenge
60
61   function verifyPreLoginChallenges (req: Request) {
62     challengeUtils.solveIfChallenges.weakPasswordChallenge, () => { return req.body.email === 'admin' + config.get('application.domain') + req.body.password }
63   }
64 }
```

```
directory listing, ensure that sensitive files are inaccessible when querying the resource.
Details: https://sg.run/DX20

249: app.use('/ftp', serveIndexMiddleware, serveIndex('ftp', { icons: true })) // vuln-code-
    snippet vuln-line directoryListingChallenge
254: app.use('/encryptionkeys', serveIndexMiddleware, serveIndex('encryptionkeys', { icons:
    true, view: 'details' }))
258: app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view:
    'details' })) // vuln-code-snippet vuln-line accessLogDisclosureChallenge

views/promotionVideo.pug
>> javascript.express.security.audit.xss.pug.explicit-unescape.template-explicit-unescape
(( Blocking ))
Detected an explicit unescape in a Pug template, using either '!' or '!{...}'. If external data can
reach these locations, your application is exposed to a cross-site scripting (XSS) vulnerability. If
you must do this, ensure no external data can reach this location.
Details: https://sg.run/3xbx

79: if (splitted.length !== 2) {

Scan Summary
✓ Scan completed successfully.
• Findings: 63 (63 blocking)
• Rules run: 307
• Targets scanned: 881
• Parsed lines: ~99.9%
• Scan skipped:
  o Files larger than files 1.0 MB: 8
  o Files matching .semgrepignore patterns: 137
  o Scan was limited to files tracked by git
  o For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 307 rules on 881 files: 63 findings.
⚡ Missed out on 1856 pro rules since you aren't logged in!
💡 Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.

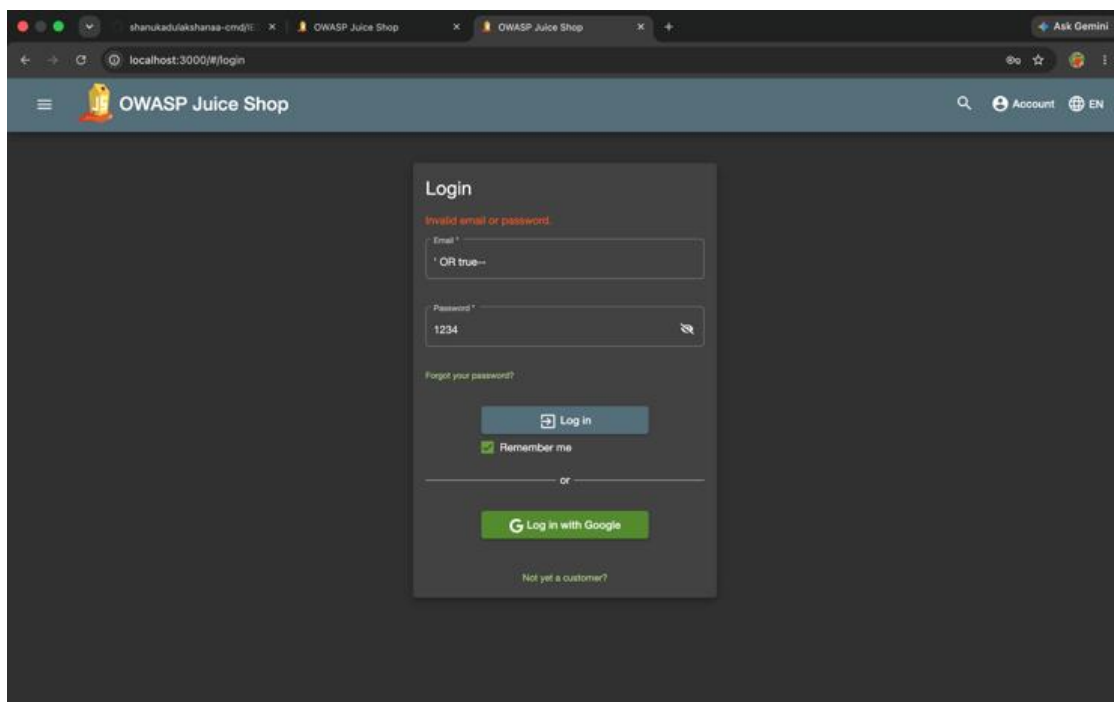
A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading

Too many findings? Try Semgrep Pro for more powerful queries and less noise.
See https://sg.run/false-positives.
shanku@shanku-MacBook-Air juice-shop %
```

```

config.yml TS login.ts M X TS web3Wallet.ts TS insecurity.ts M
routes > TS login.ts > login > <-function> > <-then() callback
20 module.exports = function login () {
41   function afterLogin (user: I Data: User, id: number //, res: Response, next: NextFunction) {
32 }
33
34 return (req: Request, res: Response, next: NextFunction) => {
35   verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
36
37   models.sequelize.query(
38     `SELECT * FROM Users WHERE email = :email AND password = :password AND deletedAt IS NULL`,
39     {
40       replacements: {
41         email: req.body.email || '',
42         password: security.hash(req.body.password || '')
43       },
44       model: UserModel,
45       plain: true
46     }
47   )
48   .then(authenticatedUser: { data: User }) => { // vuln-code-snippet neutral-line loginAdminChallenge loginBenderChallenge loginJimChallenge
49     const user = utils.queryResultToJson(authenticatedUser)
50
51     if (user.data?.id && user.data.totpSecret !== '') {
52       res.status(401).json({
53         status: 'totp_token_required',
54         data: {
55           totpToken: security.authorize({
56             userId: user.data.id,
57             type: 'password_valid_needs_second_factor_token'
58           })
59         }
60       })
61     } else if (user.data?.id) {
62       // @ts-expect-error FIXME some properties missing in user - vuln-code-snippet hide-line
63       afterLogin(user, res, next)
64     } else {
65       res.status(401).send(res.__('Invalid email or password.'))
66     }
67   }.catch((error: Error) => {
68     next(error)
69   })
70 }
71 // vuln-code-snippet end loginAdminChallenge loginBenderChallenge loginJimChallenge
72
73 function verifyPreLoginChallenges (req: Request) {

```



```
Semgrep CLI

Loading rules from registry...
Scanning 1 file (only git-tracked) with:

✓ Semgrep OSS
  ✓ Basic security coverage for first-party code vulnerabilities.

✗ Semgrep Code (SAST)
  ✗ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

✗ Semgrep Supply Chain (SCA)
  ✗ Find and fix the reachable vulnerabilities in your OSS dependencies.

💡 Get started with all Semgrep products via 'semgrep login'.
🔗 Learn more at https://sg.run/cloud.

100% 0:00:00

Scan Summary

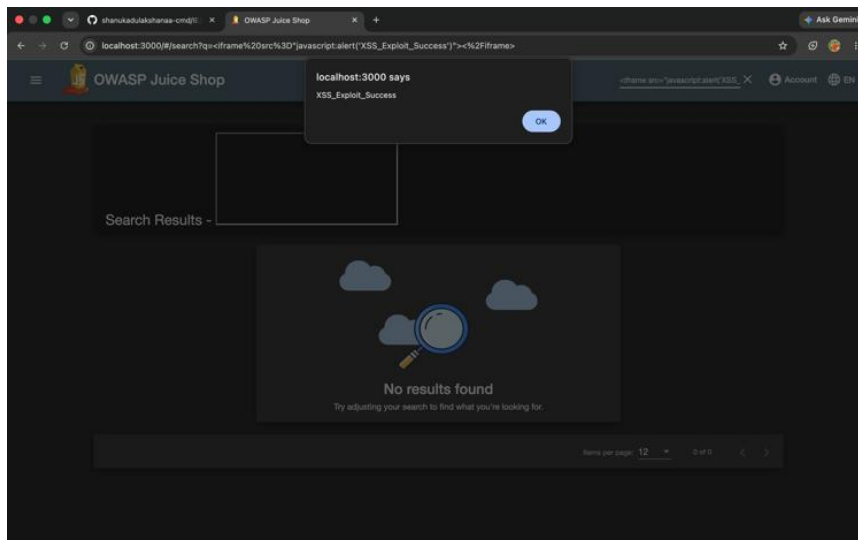
✓ Scan completed successfully.
• Findings: 0 (0 blocking)
• Rules run: 210
• Targets scanned: 1
• Parsed lines: ~100.0%
• Scan skipped:
  ◦ Not matching --include patterns: 986
  ◦ Files matching .semgrepignore patterns: 137
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 210 rules on 1 file: 0 findings.
💡 Missed out on 1856 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? 'semgrep login' for additional free Semgrep Registry rules)

📢 A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading

🔗 If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/
shanuka@shanukas-MacBook-Air juice-shop %
```

Understanding Cross-Site Scripting (XSS)

Secure Coding Evidence



```

juice-shop -- -zsh - 142x46

directory listing, ensure that sensitive files are inaccessible when querying the resource.
Details: https://sg.run/DK2G

249: app.use('/ftp', serveIndexMiddleware, serveIndex('ftp', { icons: true }))) // vuln-code-
    snippet vuln-line directoryListingChallenge
254: app.use('/encryptionkeys', serveIndexMiddleware, serveIndex('encryptionkeys', { icons:
    true, view: 'details' })))
258: app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view:
    'details' }))) // vuln-code-snippet vuln-line accessLogDisclosureChallenge

views/promotionVideo.pug
>> javascript.express.security.audit.xss.pug.explicit-unescape.template-explicit-unescape
    ({ @unescape })
    Detected an explicit unescape in a Pug template, using either '!' or '!(...)'. If external data can
    reach these locations, your application is exposed to a cross-site scripting (XSS) vulnerability. If
    you must do this, ensure no external data can reach this location.
    Details: https://sg.run/3xbs

79: if (splitted.length != 2) {

Scan Summary
✔ Scan completed successfully.
• Findings: 62 (62 blocking)
• Rules run: 387
• Targets scanned: 876
• Parsed lines: ~99.9%
• Scan skipped:
  o Files larger than files 1.0 MB: 0
  o Files matching .semgrepignore patterns: 136
  o Scan was limited to files tracked by git
  o For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 387 rules on 876 files: 62 findings.
❗ Missed out on 1856 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
📄 A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading
🔍 Too many findings? Try Semgrep Pro for more powerful queries and less noise.
See https://sg.run/false-positives.
shanku@shanku-MacBook-Air juice-shop %

```

```

! config.yml  TS login.ts 1  TS search-result.component.ts 1 X  TS insecurity.ts

frontend > src > app > search-result > TS search-result.component.ts > SearchResultComponent > filterTable
43: export class SearchResultComponent implements OnDestroy, AfterViewInit {
130:   ngOnDestroy() {
139:   }
140: }
141:
142: // vuln-code-snippet start localXssChallenge xssBonusChallenge
143: filterTable() {
144:   let queryParams: string = this.route.snapshot.queryParams.q
145:   if (queryParams) {
146:     queryParams = queryParams.trim()
147:     this.ngZone.runOutsideAngular(() => { // vuln-code-snippet hide-start
148:       this.io.socket().emit('verifyLocalXssChallenge', queryParams)
149:     }) // vuln-code-snippet hide-end
150:     this.dataSource.filter = queryParams.toLowerCase()
151:     this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParams)
152:     this.gridDataSource.subscribe((result: any) => {
153:       if (result.length === 0) {
154:         this.emptyState = true
155:       } else {
156:         this.emptyState = false
157:       }
158:     })
159:   } else {
160:     this.dataSource.filter = ''
161:     this.searchValue = undefined
162:     this.emptyState = false
163:   }
164: }
165: // vuln-code-snippet end localXssChallenge xssBonusChallenge
166:
167: startHackingInstructor (challengeName: string) {
168:   console.log('Starting instructions for challenge "${challengeName}")')
169:   import(/* webpackChunkName: "tutorial" */ './../hacking-instructor').then(module => {
170:     module.startHackingInstructorFor(challengeName)
171:   })
172: }
173:
174: showDetail (element: Product) {
175:   this.dialog.open(ProductDetailsComponent, {
176:     width: '500px',
177:     height: 'max-content',
178:     data: {
179:       productData: element

```

```

132  }
133  }
134  if (this.productSubscription) {
135    this.productSubscription.unsubscribe()
136  }
137  if (this.dataSource) {
138    this.dataSource.disconnect()
139  }
140  }
141  }
142  // vuln-code-snippet start localXssChallenge xssBonusChallenge
143  filterTable () {
144    let queryParams: string = this.route.snapshot.queryParams.q
145    if (queryParams) {
146      queryParams = queryParams.trim()
147      this.ngZone.runOutsideAngular(() => { // vuln-code-snippet hide-start
148        this.io.socket().emit('verifyLocalXssChallenge', queryParams)
149      }) // vuln-code-snippet hide-end
150      this.dataSource.filter = queryParams.toLowerCase()
151      this.searchValue = queryParams
152      this.gridDataSource.subscribe((result: any) => {
153        if (result.length == 0) {
154          this.emptyState = true
155        } else {
156          this.emptyState = false
157        }
158      })
159    } else {
160      this.dataSource.filter = ''
161      this.searchValue = undefined
162      this.emptyState = false
163    }
164  }
165  // vuln-code-snippet end localXssChallenge xssBonusChallenge
166  }
167  startHackingInstructor (challengeName: string) {
168    console.log('Starting instructions for challenge "' + challengeName + '"')
169    import(/* webpackChunkName: "tutorial" */ './../hacking-instructor').then(module => {
170      module.startHackingInstructorFor(challengeName)
171    })
172  }
173  }

```

```

2 Code Findings

frontend/src/app/search-result/search-result.component.ts
}} angular-bypass-security-trust-html
{{ (Blocking: 1)
Potential XSS: bypassSecurityTrustHtml() used with user-controlled input

125| tableData[i].description =
    this.sanitizer.bypassSecurityTrustHtml(tableData[i].description) // vuln-code-snippet
    vuln-line restfulXssChallenge
    -----
151| this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam) // vuln-code-
    snippet vuln-line localXssChallenge xssBonusChallenge

Scan Summary
✔ Scan completed successfully.
• Findings: 2 (2 blocking)
• Rules run: 1
• Targets scanned: 1
• Parsed lines: ~100.0%
• No ignore information available
Ran 1 rule on 1 file: 2 findings.

A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading
shanutka@shanutkas-MacBook-Air juice-shop %

```

Reflection on Static Finding Counts

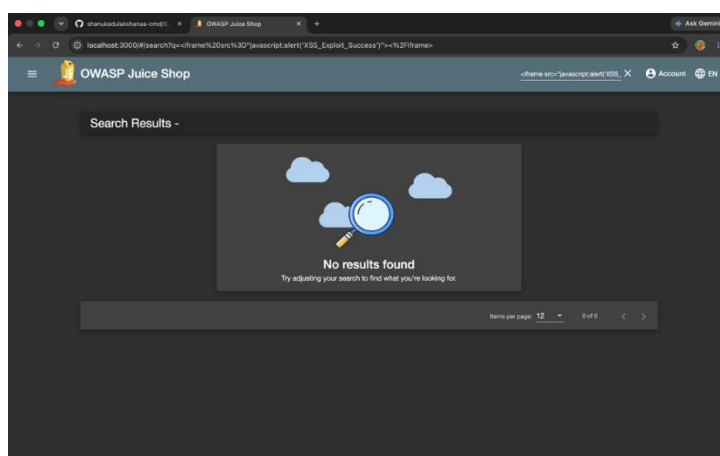


Figure: XSS Re-test After Fix

```
[shanuka@shanukas-MacBook-Air juice-shop % semgrep scan --config auto --no-git-ignore --include='*.ts' frontend/src/app/search-result/search-re|
sult.component.ts

Semgrep CLI

Loading rules from registry...
Scanning 1 file with:

✓ Semgrep OSS
  ✓ Basic security coverage for first-party code vulnerabilities.

✗ Semgrep Code (SAST)
  ✗ Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.

✗ Semgrep Supply Chain (SCA)
  ✗ Find and fix the reachable vulnerabilities in your OSS dependencies.

💡 Get started with all Semgrep products via `semgrep login`.
✦ Learn more at https://sg.run/cloud.

100% 0:00:00

Scan Summary
✓ Scan completed successfully.
  • Findings: 0 (0 blocking)
  • Rules run: 210
  • Targets scanned: 1
  • Parsed lines: ~100.0%
  • No ignore information available
Ran 210 rules on 1 file: 0 findings.
💡 Missed out on 1856 pro rules since you aren't logged in!
✦ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)

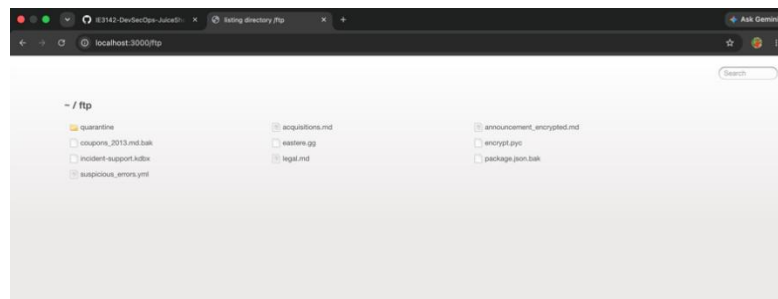
📦 A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading

✦ If Semgrep missed a finding, please send us feedback to let us know!
  See https://semgrep.dev/docs/reporting-false-negatives/
shanuka@shanukas-MacBook-Air juice-shop %
```

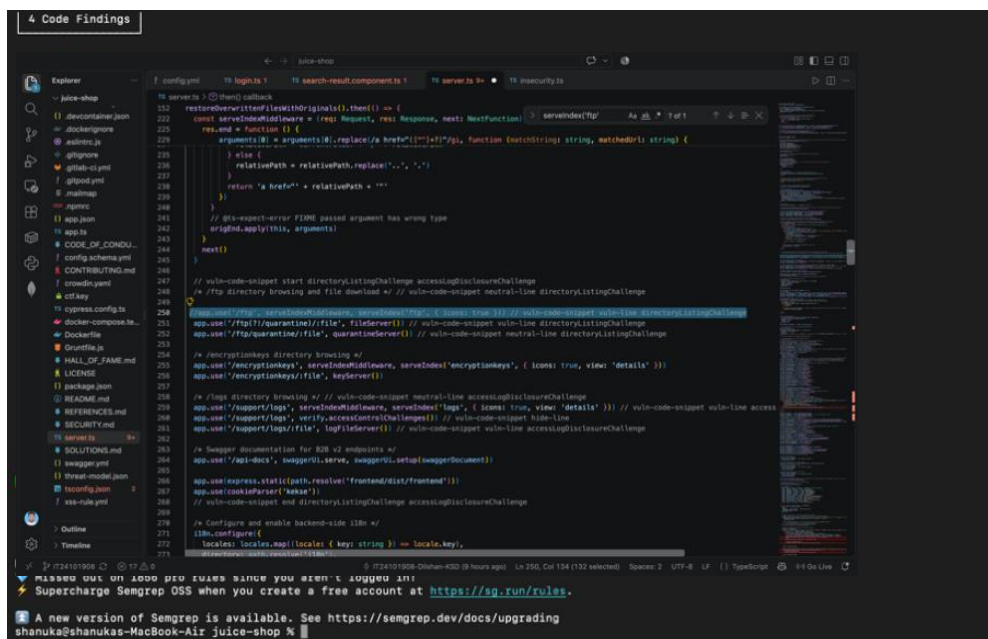
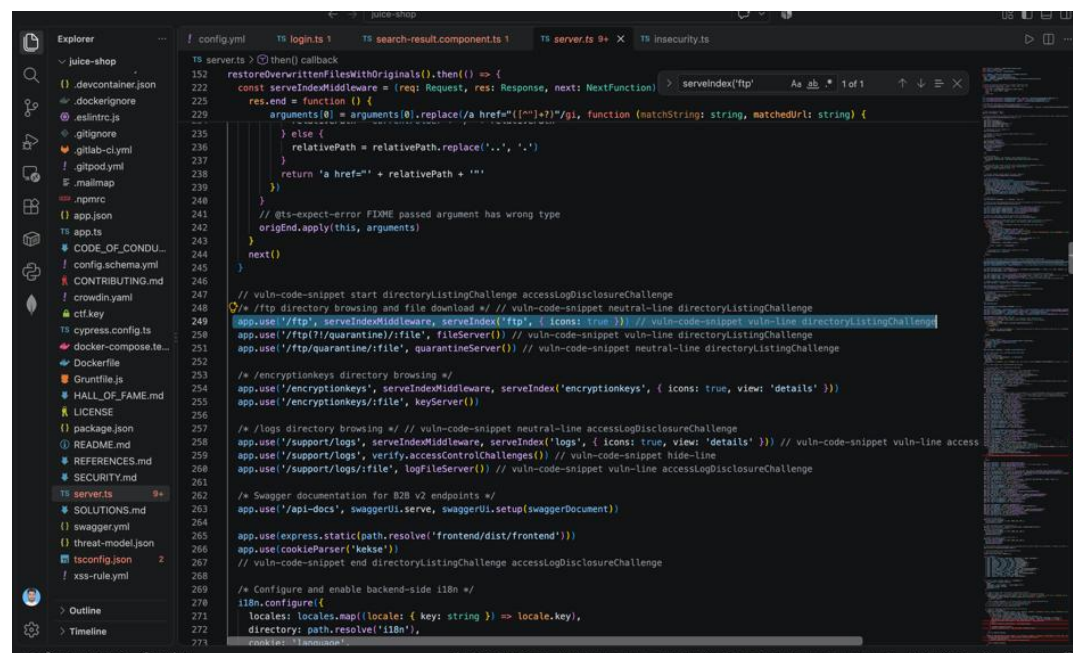
Figure: XSS – SAST Finding After Fix

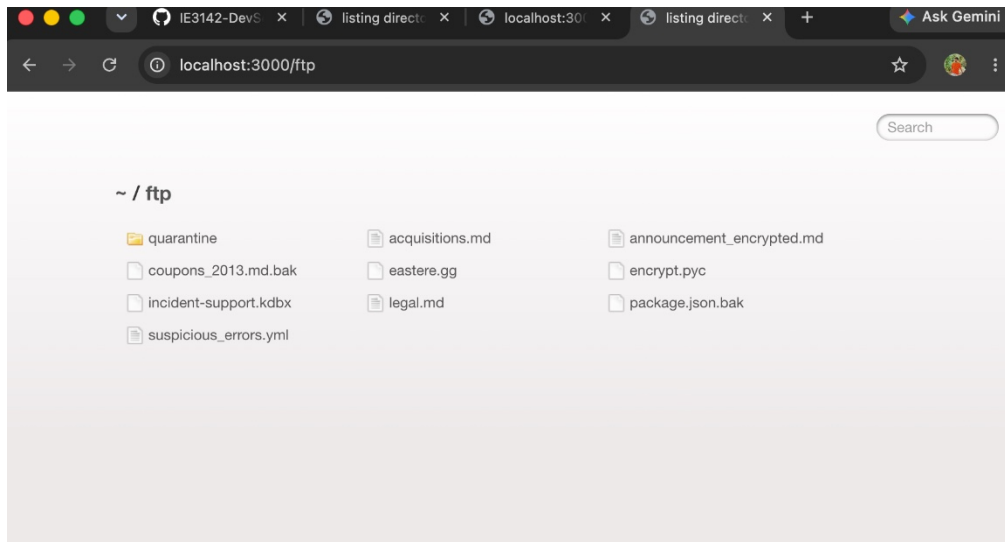
Sensitive Data Exposure (Directory Listing)

Proof of Concept (Exploit)



Vulnerability Analysis





Missing Minimum Release Age in .npmrc (Supply Chain Vulnerability)

